Team Git & Quality Assurance Guidelines

Standard Operating Procedures for Branching, Testing Tiers, and Commit Conventions

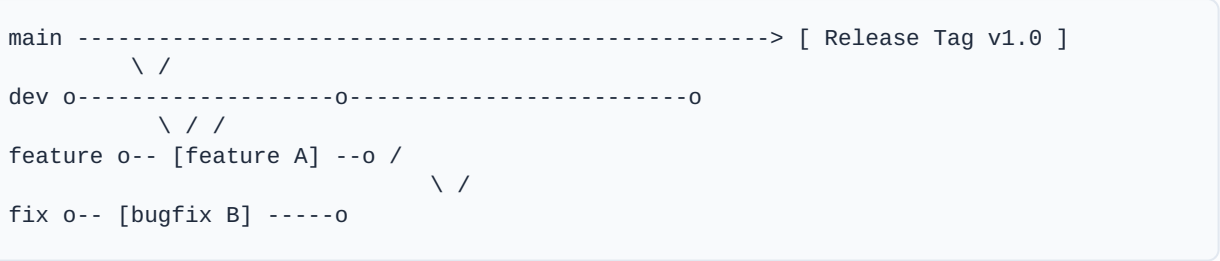
Welcome to our team development workflow! To maintain code stability, prevent integration headaches, and ensure smooth collaboration across all project members, everyone on the team must follow these guidelines.

🌿 1. Git Flow Branching Strategy

We follow the **Git Flow** branching model. No developer pushes code directly to `main` or `dev`.

Branch Structure

- **`main`**: Production release branch. Holds stable, milestone-ready, submission-safe code.
- **`dev`**: Daily integration branch. Contains all completed features that have passed review and testing.
- **`feature/*`**: Short-lived branches created for individual tasks or user stories (e.g., `feature/user-auth`, `feature/order-api`).
- **`fix/*`** **or** **`bugfix/*`**: Short-lived branches created to resolve bugs (e.g., `fix/null-pointer-exception`).



Step-by-Step Developer Workflow

Step 1: Branch off `dev`

Always ensure your local `dev` is up-to-date before starting a new task:

```
git checkout dev
git pull origin dev
git checkout -b feature/your-feature-name
```

Step 2: Work and Commit

Make frequent, small commits on your local branch following our commit conventions:

```
git add .
git commit -m "feat: implement user registration endpoint"
```

Step 3: Keep Branch Updated

Periodically pull changes from `dev` to avoid huge merge conflicts later:

```
git checkout feature/your-feature-name
git merge dev
```

Step 4: Push Branch & Open PR
Push your feature branch to GitHub and open a Pull Request (PR) **targeting** `dev`:

```
git push -u origin feature/your-feature-name
```

🛡️ 2. The 3-Tier Quality Assurance (QA) System

QA is not a separate phase done at the end—it is built into every stage of development.

```
+--------------------+ +----------------------+ +-----------------------+
| 1. Pre-Commit (Local)| ----> | 2. PR Review (GitHub) | ----> | 3. Integration (Dev) |
| Unit Tests / Lint | | CI Build / Peer Test | | E2E / System Testing |
+--------------------+ +----------------------+ +-----------------------+
```

Tier	When It Happens	Responsibility	Focus & Scope	Verification Method
Tier 1: Pre-Commit	Local machine during coding	Authoring Developer	Unit tests, static code analysis, linting, syntax validation.	Developer runs test suite locally before pushing.
Tier 2: PR Review	On opening/ updating a PR	GitHub Actions (CI) + Assigned Reviewer	Automated build check, test execution, manual code review, edge-case testing.	Automated CI check turns green + 1 peer review approval.
Tier 3: Integration	Shared `dev` branch	Whole Team / Lead	End-to-End (E2E) testing, cross-feature compatibility, database migrations.	Manual system testing on integrated build before releasing to `main`.

📝 3. Git Commit Conventions

We follow Conventional Commits to keep our commit history clean, readable, and easy to audit.

Commit Format

```
<type>: <short description in imperative mood>
```

Allowed Types

- `feat`: A new feature for the user or application.
- `fix`: A bug fix.
- `docs`: Documentation changes only (e.g., README, inline code docs).
- `style`: Formatting, missing semicolons, white space (no code logic changes).
- `refactor`: Code restructuring that neither fixes a bug nor adds a feature.
- `test`: Adding missing tests or refactoring existing tests.
- `chore`: Maintenance tasks, updating dependencies, or build tool configurations.

Commit Examples

- ✅ `feat: add payment gateway checkout endpoint`

- ✅ `fix: resolve CORS issue on user login API`
- ✅ `docs: update setup instructions in README`
- ❌ `fixed stuff` *(vague and unstructured)*
- ❌ `feat: Added the new login page UI layout` *(use lowercase imperative "add" instead of "Added")*

💡 4. Team Best Practices & Golden Rules

1. **Never Push Directly to `main` or `dev`**: All additions to integrated branches must pass through a Pull Request with at least **1 review approval**.
2. **One Feature, One Branch**: Keep feature branches small and focused. Smaller PRs get reviewed faster and carry fewer merge conflicts.
3. **Self-Review Before Requesting**: Before assigning a teammate to review your PR, inspect your own diff on GitHub to clean up commented-out code, debug logs, or extra files.
4. **Clean Up Merged Branches**: Delete your feature branch on GitHub immediately after it has been merged into `dev`.
5. **Ask Before Forcing**: Never execute `git push --force` or `git reset --hard` on shared branches (`dev` or `main`). If you run into a confusing Git error, alert the group chat first!